



Hands-on

Lab 2 – Modelo de Grafos

No Sandbox Neo4j selecionar Blank Sandbox Graph Data Science. Observe como tem vários exemplos já construídos para exploração futura e mais detalhes podem ser vistos em <https://neo4j.com/graphgists>.

Select a project

This will create a new Neo4j database instance

Your own data



Pre Built Data

Blank Sandbox

Blank Sandbox. Load your own data with LOAD CSV or create data from scratch.

[+ Launch Project](#)

Nosso projeto será criado e teremos três dias para realizarmos comandos neste ambiente. Para isto abrimos uma interface gráfica no browser (Open with Browser).

Name	Status	
Blank Sandbox	Running Expires in about 3 days	Open Open with Browser Open with Bloom Open with NEuler

Centralidade

MATCH (n)
DETACH DELETE n

Podemos criar um grafo a partir de arquivos csv. Neste exemplo usuários e seus seguidores.

```
WITH "https://github.com/neo4j-graph-
analytics/book/raw/master/data/" AS base
WITH base + "social-nodes.csv" AS uri
LOAD CSV WITH HEADERS FROM uri AS row
MERGE (:User {id: row.id})
```

```
WITH "https://github.com/neo4j-graph-
analytics/book/raw/master/data/" AS base
WITH base + "social-relationships.csv" AS uri
LOAD CSV WITH HEADERS FROM uri AS row
MATCH (source:User {id: row.src})
MATCH (destination:User {id: row.dst})
MERGE (source)-[:FOLLOWS]->(destination)
```

*/// centralidade de intermediação - **betweenness** centrality*

```
CALL gds.graph.project ('Fiap_BC', 'User', 'FOLLOWS')
```

```
CALL gds.betweenness.stream('Fiap_BC')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS name, score
ORDER BY score DESC
```

/// vamos adicionar um novo user o Jason

```
WITH ["James", "Michael", "Alice", "Doug", "Amy"] AS
existingUsers
MATCH (existing:User) WHERE existing.id IN existingUsers
MERGE (newUser:User {id: "Jason"})
MERGE (newUser)<-[:FOLLOWS]-(existing)
MERGE (newUser)-[:FOLLOWS]->(existing)
```

```
CALL gds.graph.drop('Fiap_BC') YIELD graphName;
```

```
CALL gds.graph.project('Fiap_BC', 'User', 'FOLLOWS')
```

```
CALL gds.betweenness.stream('Fiap_BC')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS name, score
ORDER BY name ASC
```

/// centralidade de proximidade – closeness

```
CALL gds.graph.project('Fiap_Close', 'User', 'FOLLOWS')
```

```
CALL gds.beta.closeness.stream('Fiap_Close')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS id, score
ORDER BY score DESC
```

/// Pagerank

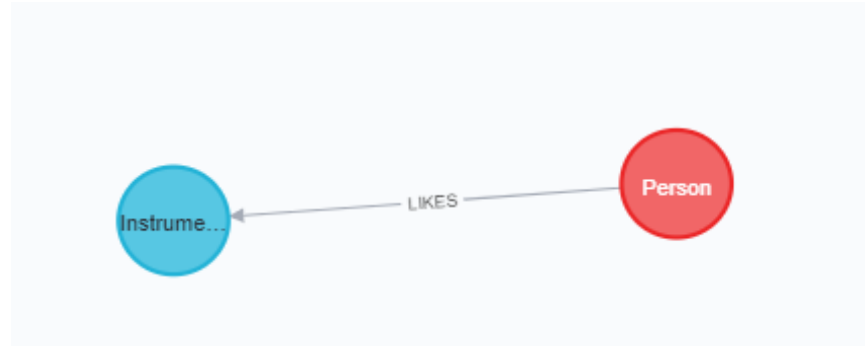
```
CALL gds.graph.project ("Fiap_PR", "User", "FOLLOWS");

CALL gds.pageRank.stream("Fiap_PR",
    {maxIterations: 20, dampingFactor: 0.85}) YIELD nodeId,
score
RETURN gds.util.asNode(nodeId).id AS name, score
```

ORDER BY score DESC LIMIT 10

Exemplos Similaridade

Comandos para criação do grafo representando nos nós as Pessoas e os Instrumentos musicais e nas arestas qual pessoa gosta de qual instrumento.



Comandos para criação:

```
CREATE
  (alice:Person {name: 'Alice'}),
  (bob:Person {name: 'Bob'}),
  (carol:Person {name: 'Carol'}),
  (dave:Person {name: 'Dave'}),
  (eve:Person {name: 'Eve'}),
  (guitar:Instrument {name: 'Guitar'}),
  (synth:Instrument {name: 'Synthesizer'}),
  (bongos:Instrument {name: 'Bongos'}),
  (trumpet:Instrument {name: 'Trumpet'}),

  (alice)-[:LIKES]->(guitar),
  (alice)-[:LIKES]->(synth),
  (alice)-[:LIKES {strength: 0.5}]->(bongos),
  (bob)-[:LIKES]->(guitar),
  (bob)-[:LIKES]->(synth),
  (carol)-[:LIKES]->(bongos),
  (dave)-[:LIKES]->(guitar),
  (dave)-[:LIKES]->(synth),
  (dave)-[:LIKES]->(bongos);
```

Similaridade no grafo.

//obter similaridade entre as pessoas com base nos instrumentos que elas gostam

```
CALL gds.graph.project (
  'Fiap_Sim',
  ['Person', 'Instrument'],
  {
    LIKES: {
      type: 'LIKES',
      properties: {
        strength: {
          property: 'strength',
          defaultValue: 1.0
        }
      }
    }
  }
);
```

```
CALL gds.nodeSimilarity.stream('Fiap_Sim')
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1,
gds.util.asNode(node2).name AS Person2, similarity
ORDER BY similarity DESCENDING, Person1, Person2
```

//algoritmo K-Nearest Neighbors a distância é calculada com base nas propriedades do nó

Atribuir a idade para cada Pessoa

```
CREATE (alice:Person {name: 'Alice', age: 24})
CREATE (bob:Person {name: 'Bob', age: 73})
CREATE (carol:Person {name: 'Carol', age: 24})
CREATE (dave:Person {name: 'Dave', age: 48})
CREATE (eve:Person {name: 'Eve', age: 67});
```

```
CALL gds.graph.project (
  'Fiap_KN',
  {
    Person: {
      label: 'Person',
      properties: 'age'
    }
  },
  '*'
);
```

```
CALL gds.knn.stream('Fiap_KN', {
  topK: 1,
  nodeProperties: ['age'],
  randomSeed: 1337,
  concurrency: 1,
  sampleRate: 1.0,
  deltaThreshold: 0.0
})
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS Person1,
gds.util.asNode(node2).name AS Person2, similarity
ORDER BY similarity DESCENDING, Person1, Person2
```

Comunidades

```
MATCH (n)
DETACH DELETE n;
```

```
CREATE (a:Node {name: 'Antonia'}),
      (b:Node {nome: 'Bernardo'}),
      (c:Node { nome: 'Claudia'}),
      (d:Node { nome: 'Dioneia'}),
      (e:Node { nome: 'Eurico'}),
      (f:Node { nome: 'Fabiano'}),
      (a)-[:REL {nota: 50}]->(b),
      (a)-[:REL { nota: 50}]->(c),
      (a)-[:REL { nota: 100}]->(d),
      (b)-[:REL { nota: 40}]->(d),
      (c)-[:REL { nota: 40}]->(d),
      (c)-[:REL { nota: 80}]->(e),
      (d)-[:REL { nota: 30}]->(e),
      (d)-[:REL { nota: 80}]->(f),
      (e)-[:REL { nota: 40}]->(f);

CALL gds.graph.project(
  'Comunidade_L',
  'Node',
  {
    REL: {
      orientation: 'UNDIRECTED'
    }
  }
);
```

```
CALL gds.louvain.stream('Comunidade_L')
YIELD nodeId, communityId
RETURN gds.util.asNode(nodeId).nome AS nome, communityId
ORDER BY communityId, nome ASC;
```

```
CALL gds.louvain.stats('Comunidade_L')
YIELD communityCount;
```



```
CALL gds.graph.project(  
  'Comunidade_LP',  
  'Node',  
  {  
    REL: {  
      orientation: 'UNDIRECTED'  
    }  
  },  
  {  
    relationshipProperties: 'nota'  
  }  
);
```

```
CALL gds.louvain.stream('Comunidade_LP', {  
  relationshipWeightProperty: 'nota' })  
YIELD nodeId, communityId, intermediateCommunityIds  
RETURN gds.util.asNode(nodeId).nome AS nome, communityId,  
intermediateCommunityIds  
ORDER BY nome ASC
```

Predição de Links

```
MATCH (n)
DETACH DELETE n;
```

```
CREATE (a:Node {name: 'A', community: 1}),
      (b:Node {name: 'B', community: 1}),
      (c:Node {name: 'C', community: 1}),
      (d:Node {name: 'D', community: 1}),
      (e:Node {name: 'E', community: 5}),
      (f:Node {name: 'F', community: 5}),
      (a)-[:REL]->(b),
      (a)-[:REL]->(c),
      (a)-[:REL]->(d),
      (b)-[:REL]->(d),
      (c)-[:REL]->(d),
      (c)-[:REL]->(e),
      (d)-[:REL]->(e),
      (d)-[:REL]->(f),
      (e)-[:REL]->(f);
```

```
MATCH (p1:Node {name: 'A'})
MATCH (p2:Node) WHERE p1 <> p2
RETURN p2.name AS name,
      gds.alpha.linkprediction.adamicAdar(p1, p2) AS aA,
      gds.alpha.linkprediction.commonNeighbors(p1, p2) AS cN,
      gds.alpha.linkprediction.preferentialAttachment(p1, p2) AS
pA,
      gds.alpha.linkprediction.resourceAllocation(p1, p2) AS rA,
      gds.alpha.linkprediction.sameCommunity(p1, p2) AS sC,
      gds.alpha.linkprediction.totalNeighbors(p1, p2) AS tN;
```

```
MATCH (p1:Node {name: 'A'})
MATCH (p2:Node) WHERE p1 <> p2
RETURN p2.name AS name,
      gds.alpha.linkprediction.adamicAdar(p1, p2) AS score order by
score desc;
```

```
MATCH (p1:Node {name: 'A'})
MATCH (p2:Node) WHERE p1 <> p2
RETURN p2.name AS name,
       gds.alpha.linkprediction.preferentialAttachment(p1, p2) AS
score;
```

```
MATCH (p1:Node {name: 'A'})
MATCH (p2:Node) WHERE p1 <> p2
RETURN p2.name AS name,
       gds.alpha.linkprediction.resourceAllocation(p1, p2) AS score;
```

```
MATCH (p1:Node {name: 'A'})
MATCH (p2:Node) WHERE p1 <> p2
RETURN p2.name AS name,
       gds.alpha.linkprediction.sameCommunity(p1, p2) AS score;
```

```
MATCH (p1:Node {name: 'A'})
MATCH (p2:Node) WHERE p1 <> p2
RETURN p2.name AS name,
       gds.alpha.linkprediction.totalNeighbors(p1, p2) AS score;
```

para deletar as projeções

```
CALL gds.graph.drop('Comunidade_LP') YIELD graphName;
```

Caminhos

```
CREATE (LocationA:Location { name: "Location A" })
CREATE (LocationB:Location { name: "Location B" })
CREATE (LocationC:Location { name: "Location C" })
CREATE (LocationD:Location { name: "Location D" })
CREATE (LocationE:Location { name: "Location E" })
CREATE (LocationF:Location { name: "Location F" })
CREATE (LocationG:Location { name: "Location G" })
CREATE (LocationH:Location { name: "Location H" })
CREATE (LocationI:Location { name: "Location I" })

CREATE
  (LocationA)-[:CONNECTED_TO { distance: 5 }]->(LocationB),
  (LocationB)-[:CONNECTED_TO { distance: 6 }]->(LocationC),
  (LocationC)-[:CONNECTED_TO { distance: 4 }]->(LocationI),
  (LocationA)-[:CONNECTED_TO { distance: 3 }]->(LocationD),
  (LocationD)-[:CONNECTED_TO { distance: 4 }]->(LocationE),
  (LocationE)-[:CONNECTED_TO { distance: 5 }]->(LocationI),
  (LocationA)-[:CONNECTED_TO { distance: 2 }]->(LocationF),
  (LocationF)-[:CONNECTED_TO { distance: 3 }]->(LocationG),
  (LocationG)-[:CONNECTED_TO { distance: 2 }]->(LocationH),
  (LocationH)-[:CONNECTED_TO { distance: 1 }]->(LocationI);
```

encontrar a distância mais curta do Local A ao Local I. Como todas as relações já estão definidas, percorrer o caminho e somar as distâncias ao longo do caminho até o destino.

```
MATCH (from:Location { name:"Location A" }), (to:Location {
name: "Location I"}) , path = (from)-[:CONNECTED_TO*]->(to)
RETURN path AS shortestPath,
  reduce(distance = 0, r in relationships(path) |
distance+r.distance) AS totalDistance
ORDER BY totalDistance ASC
LIMIT 1
```

obter o caminho mais longo

```
MATCH (from:Location { name:"Location A" }), (to:Location {
name: "Location I"}) , path = (from)-[:CONNECTED_TO*]->(to)
RETURN path AS longestPath,
```

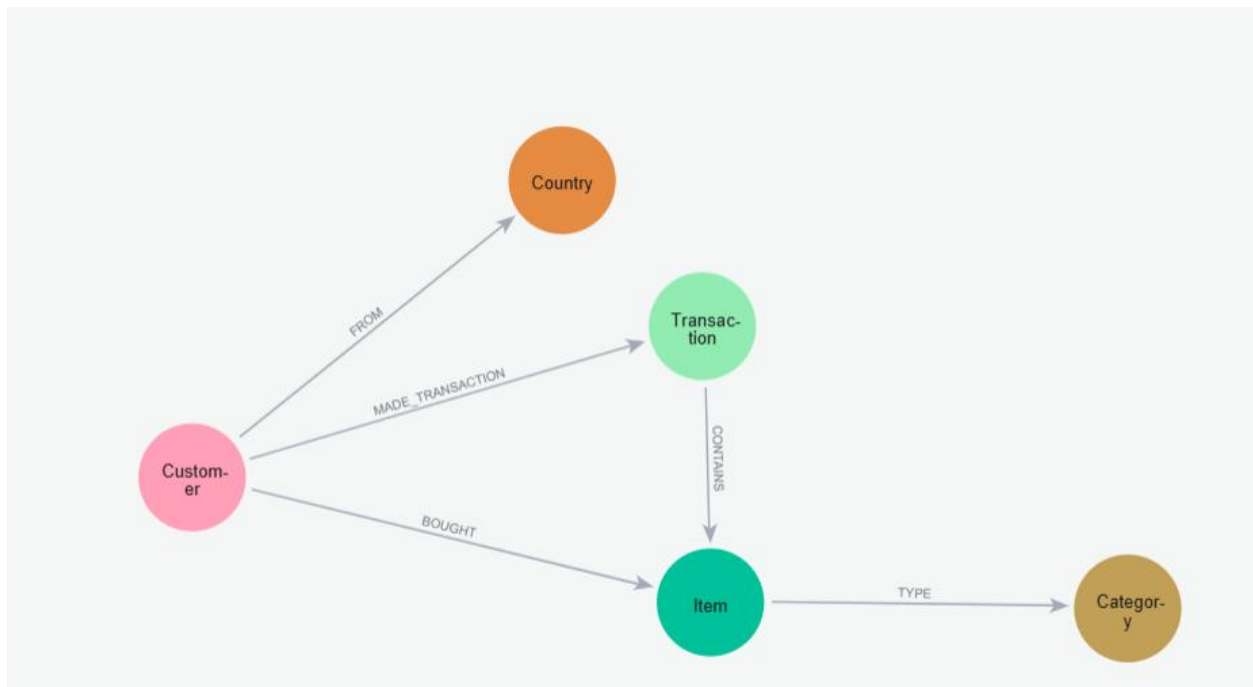
```
    reduce(distance = 0, r in relationships(path) |  
distance+r.distance) AS totalDistance  
    ORDER BY totalDistance DESC  
    LIMIT 1
```

CASE VAREJO

1. Criar uma base de dados a partir do script load_data_transacoes.txt

Após carga verificar o modelo

```
CALL db.schema.visualization()
```



1) Explorar os dados respondendo:

Total de itens, de categorias, de clientes, de países e de transações

Quais os dez clientes com o maior número de transações e quais os produtos mais comprados?

Total de itens por categoria

Total de clientes por país

2. Realizar Segmentação de Clientes

- a. Similaridade por Cliente por Categoria – criar projeção
Customer_Similarity_Graph_Category

```
CALL gds.graph.project(
  'Customer_Similarity_Graph_Category',
  ['Customer', 'Category', 'Transaction'],
  {
    MADE_TRANSACTION: {
      orientation: 'UNDIRECTED'
    },
    CONTAINS: {
      orientation: 'UNDIRECTED'
    },
    TYPE: {
      orientation: 'UNDIRECTED'
    }
  }
);
```

- b. Similaridade por Cliente por Categoria – gravando relacionamento Similar

```
CALL
gds.nodeSimilarity.mutate('Customer_Similarity_Graph_Category',
{mutateRelationshipType:'Similar',
mutateProperty:'score',topK:10, similarityCutoff:0.25})
```

2) Explicar o comando mutate e destacar os dez clientes relacionados

- c. Louvain comunidade formada a partir do relacionamento Similar e propriedade score

```
CALL
gds.louvain.stream('Customer_Similarity_Graph_Category',{relationshipTypes:['Similar'], relationshipWeightProperty:'score'})
YIELD nodeId, communityId RETURN communityId, count(nodeId) as
size ORDER by size DESC
```

3) Quantas comunidades foram criadas com o total de clientes em cada uma?

d. Eliminar projeção

```
CALL gds.graph.drop('Customer_Similarity_Graph_Category')
```

e. Cliente similaridade por item

```
CALL gds.graph.project(  
  'Customer_Similarity_Graph_Items',  
  ['Customer','Item'],  
  'BOUGHT'  
)
```

f. Node similaridade

```
CALL  
gds.nodeSimilarity.mutate('Customer_Similarity_Graph_Items',  
{mutateRelationshipType:'Similar',  
mutateProperty:'score',topK:10, similarityCutoff:0.1});
```

4) Destacar os dez itens relacionados

g. Louvain

```
CALL  
gds.louvain.stream('Customer_Similarity_Graph_Items',{relationshipsTypes:['Similar'], relationshipWeightProperty:'score'}) YIELD  
nodeId, communityId RETURN communityId, count(nodeId) as size  
ORDER by size DESC
```

h. Louvain Mutate


```
CALL
gds.louvain.mutate('Customer_Similarity_Graph_Items',{relationshipsTypes:['Similar'], relationshipWeightProperty:'score',
mutateProperty:'louvainCommunity'});
```

5) Quantas comunidades foram criadas com o total de itens em cada uma?

```
CALL
gds.graph.writeNodeProperties('Customer_Similarity_Graph_Items',
['louvainCommunity']);
CALL
gds.louvain.stream('Customer_Similarity_Graph_Items',{relationshipsTypes:['Similar'], relationshipWeightProperty:'score'}) YIELD
nodeId, communityId RETURN communityId, count(nodeId) as size
ORDER by size DESC
```

i. Eliminar a projeção

```
CALL gds.graph.drop('Customer_Similarity_Graph_Items');
```

3. Item Similaridade

a. Item similaridade = 6115 (escolher comunidade existente em clientes)

```
CALL gds.graph.project.cypher(
'item_copurchase_6115',

// Nodes
'
MATCH (i:Item) <--(c:Customer)
where c.louvainCommunity = 6115
return distinct id(i) as id
',

// Relationships
'
MATCH (i1:Item) <--(c:Customer) --> (i2:Item)
```

```
WHERE c.louvainCommunity = 6115
and id(i1) < id(i2)
RETURN id(i1) as source, id(i2) as target,
count(c) as weight
'
);
```

b. Gravar relacionamento similaridade

```
CALL gds.nodeSimilarity.mutate('item_copurchase_6115',
{mutateRelationshipType:'SIMILAR_6115',
mutateProperty:'score',topK:10, similarityCutoff:0.1});
```

c. Item closeness

```
CALL gds.closeness.write(
    'item_copurchase_6115',
    {
        writeProperty: 'Closeness_Centrality_Community_6115'
    }
);
```

d. Item Pagerank

```
CALL gds.pageRank.write(
    'item_copurchase_6115',
    {
        relationshipWeightProperty: 'weight',
        writeProperty: 'PageRank_Community_6115'
    }
);
```

```
MATCH (i:Item)<--(c:Customer)
WHERE c.louvainCommunity = 6115
RETURN DISTINCT i
LIMIT 10;
```

```
MATCH (i:Item)<--(c:Customer)
```

```
WHERE c.louvainCommunity = 6115
RETURN DISTINCT i.PageRank_Community_6115,
i.Closeness_Centrality_Community_6115
LIMIT 10;
```

```
MATCH p=(i1:Item)-[r:SIMILAR_6115]->(i2)
WHERE r.score > .1 AND r.score < .8
RETURN i1.Description, i2.Description, r.score as score
order by score desc
```

Encontrar similares

```
CALL gds.nodeSimilarity.stream('item_copurchase_6115', { topK:
1, topN: 10 })
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).Description AS Produto1,
gds.util.asNode(node2).Description AS Produto2, similarity
ORDER BY similarity DESC, Produto1, Produto2
```

6. Listar os itens com todos os scores calculados dentro da comunidade escolhida